0.71. La croix en une ligne : deplace_croix

deplace_croix(x, y, tuile, vitesse), fonction de la console : tout le bloc de la croix du 0.66 en une seule ligne, dans la boucle, à la vitesse qu'on choisit.

1. Ce qu'il faut comprendre

C'est le programme du 0.66, avec un seul changement : tout le bloc qui lisait la croix (l'attente, l'effacement, les quatre `if`, le `poser`) est remplacé par **une ligne**.

Ce qui est nouveau ici : `deplace_croix(x, y, tuile, vitesse)`**.** C'est une fonction **de la console**, comme `ALPHABET` est un tableau de la console : on s'en sert **sans rien déclarer**, et elle n'est ajoutée à la cartouche que si on l'appelle.

La vitesse, 4e réglage, est obligatoire : c'est le temps entre deux pas, en **millisecondes**. `250` : un pas tous les quarts de seconde, **4 cases par seconde**. Plus le nombre est petit, plus la lettre va vite (le 0.72 essaie `100`). Il s'écrit **en clair** : le compilateur le traduit en images avant le jeu, comme `ms(250)` = 15 images.

Ce qu'elle fait, à chaque appel : elle lit la croix ; si une flèche est tenue et que le temps d'attente est passé, elle fait **un pas** d'une case (en restant dans l'écran), efface l'ancienne case, et attend le temps de la vitesse avant le pas suivant. Puis elle pose la lettre à sa place.

Elle ne bloque pas : un appel, au plus un pas, et elle rend la main. On l'appelle donc **dans la boucle**, une fois par image, après `image()`.

Pas de `x =` : comme `deplace` au 0.26, elle est seule sur sa ligne, et la console range la nouvelle colonne dans `x` et la nouvelle ligne dans `y`.

Elle ne clignote pas : elle n'efface l'ancienne case **que si la lettre a bougé**. Immobile, la lettre est simplement reposée à la même place.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <deplace_croix>` → 0.66.3. La fonction deplace_croix() — suivre la croix, case par case
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <deplace_croix>  // une tuile qui suit la croix, case par case`	Demande à la console « deplace_croix » (ce que c'est : une tuile qui suit la croix, case par case). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui suit la croix, case par case ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`uint8_t x = 9;  // la COLONNE de la lettre : elle part du milieu`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 9. (Le commentaire le dit : « la COLONNE de la lettre : elle part du milieu ».)

`uint8_t y = 8;  // la LIGNE de la lettre`	Une variable : une boîte appelée y, qui retient un nombre de 0 à 255, et qui commence à 8. (Le commentaire le dit : « la LIGNE de la lettre ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();  // une fois par image, 60 fois par seconde`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « une fois par image, 60 fois par seconde ».)
`// La ligne qui remplace tout le bloc du 0.66 :`	Un commentaire : une phrase pour celui qui lit (« La ligne qui remplace tout le bloc du 0.66 : »). La console l'ignore.
`//   deplace_croix(x, y, ALPHABET[0], 250);`	Un commentaire : une phrase pour celui qui lit (« deplace_croix(x, y, ALPHABET[0], 250); »). La console l'ignore.
`//  |  |  |  |`	Un commentaire : une phrase pour celui qui lit (« |||| »). La console l'ignore.
`//  |  |  |  +-- vitesse : 250 ms entre deux pas,`	Un commentaire : une phrase pour celui qui lit (« ||| +-- vitesse : 250 ms entre deux pas, »). La console l'ignore.
`//  |  |  |  soit 4 cases par seconde`	Un commentaire : une phrase pour celui qui lit (« ||| soit 4 cases par seconde »). La console l'ignore.
`//  |  |  +--------------- tuile : ce qui bouge, la lettre A`	Un commentaire : une phrase pour celui qui lit (« || +--------------- tuile : ce qui bouge, la lettre A »). La console l'ignore.
`//  +--+------------------ x, y : où elle est ; la console`	Un commentaire : une phrase pour celui qui lit (« +--+------------------ x, y : où elle est ; la console »). La console l'ignore.
`//  y range sa nouvelle place`	Un commentaire : une phrase pour celui qui lit (« y range sa nouvelle place »). La console l'ignore.
`// Ce qu'elle fait pour nous, à chaque image :`	Un commentaire : une phrase pour celui qui lit (« Ce qu'elle fait pour nous, à chaque image : »). La console l'ignore.
`//   - lit la croix (DROITE, GAUCHE, HAUT, BAS)`	Un commentaire : une phrase pour celui qui lit (« - lit la croix (DROITE, GAUCHE, HAUT, BAS) »). La console l'ignore.
`//   - au plus UN pas d'une case, puis attend 250 ms`	Un commentaire : une phrase pour celui qui lit (« - au plus UN pas d'une case, puis attend 250 ms »). La console l'ignore.
`//   - reste dans l'écran (colonnes 0 à 19, lignes 0 à 17)`	Un commentaire : une phrase pour celui qui lit (« - reste dans l'écran (colonnes 0 à 19, lignes 0 à 17) »). La console l'ignore.
`//   - efface l'ancienne case, SEULEMENT si la lettre a bougé`	Un commentaire : une phrase pour celui qui lit (« - efface l'ancienne case, SEULEMENT si la lettre a bougé »). La console l'ignore.
`//   - pose la lettre à sa place`	Un commentaire : une phrase pour celui qui lit (« - pose la lettre à sa place »). La console l'ignore.
`deplace_croix(x, y, ALPHABET[0], 250);`	Appel de la fonction deplace_croix de la console, avec les arguments x, y, ALPHABET[0], 250. Ce qu'elle fait : la lettre suit la croix, case par case : à appeler dans la boucle, après image(). vitesse : le temps entre deux pas, en ms écrites en clair (250 = 4 cases par seconde, 100 = 10). Reste dans l'écran, efface seulement si elle a bougé (pas de clignotement). Seule sur sa ligne : deplace_croix(x, y, ALPHABET[0], 250); range la place dans x et y.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1181 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage         73 o  ← la console, à l'allumage  — la mise en route de la console
  deplace_croix()  217 o  ← main()  — une fonction fournie par le langage
  main()            32 o  ← démarrage  — ta fonction
  VBlank             8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage     28 o  ← image()  — attend l'image suivante
  AttendreVBlank    16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces     12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase       10 o  ← effacer(), poser()  — trouve l'adresse d'une case de l'écran
  EffacerCases      12 o  ← effacer()  — efface des cases
  CopierTuiles      18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte      14 o  ← démarrage  — vide les deux cartes de l'écran
  LireManette       37 o  ← bouton()  — lit les huit boutons
  Tuiles           704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
DonneesTransfert, Notes
Fonctions du langage appelées :
  bouton()        × 4    20 o sur place, dans deplace_croix()  + LireManette (37 o)
  effacer()       × 1    15 o sur place, dans deplace_croix()  + AdresseCase (10 o), EffacerCases (12 o)
  poser()         × 1    13 o sur place, dans deplace_croix()  + AdresseCase (10 o)
  image()         × 1     3 o sur place, dans main()  + AttendreImage (28 o)
  images()        × 1     2 o sur place, dans deplace_croix()
Tes fonctions appelées :
  deplace_croix() × 1    19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_croix>   // une tuile qui suit la croix, case par case
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 9;        // la COLONNE de la lettre : elle part du milieu
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;        // la LIGNE de la lettre
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

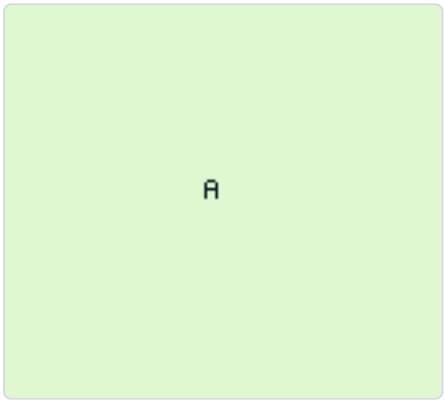
```
int main() {
  while (true) {
    image();          // une fois par image, 60 fois par seconde

    // La ligne qui remplace tout le bloc du 0.66 :
    //
    //   deplace_croix(x, y, ALPHABET[0], 250);
    //                 |  |  |           |
    //                 |  |  |           +-- vitesse : 250 ms entre deux pas,
    //                 |  |  |                         soit 4 cases par seconde
    //                 |  |  |
    //                 |  |  +--------------- tuile   : ce qui bouge, la lettre A
    //                 +--+------------------ x, y    : où elle est ; la console
    //                                                   y range sa nouvelle place
    //
    // Ce qu'elle fait pour nous, à chaque image :
    //   - lit la croix (DROITE, GAUCHE, HAUT, BAS)
    //   - au plus UN pas d'une case, puis attend 250 ms
    //   - reste dans l'écran (colonnes 0 à 19, lignes 0 à 17)
    //   - efface l'ancienne case, SEULEMENT si la lettre a bougé
    //   - pose la lettre à sa place
    deplace_croix(x, y, ALPHABET[0], 250);
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Un A au milieu de l'écran, qui se déplace case par case avec les flèches, 4 cases par seconde. Le programme tient en quelques lignes.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A part du milieu (9, 8)

✓ 60 images à droite : 4 pas, un tous les quarts de seconde (x = 13)

✓ la flèche bas aussi : y rangé tout seul (y = 10)

✓ il est à sa nouvelle place

✓ et plus au départ

Ensuite : **0.71.1. La croix en une ligne : deplace_croix — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++